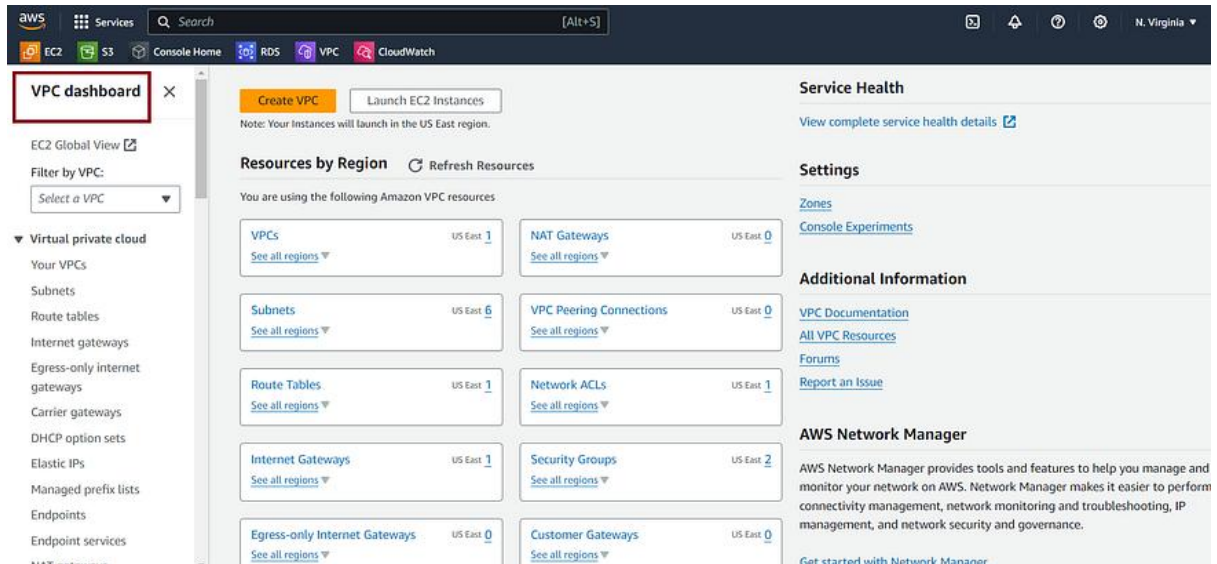


# VPC

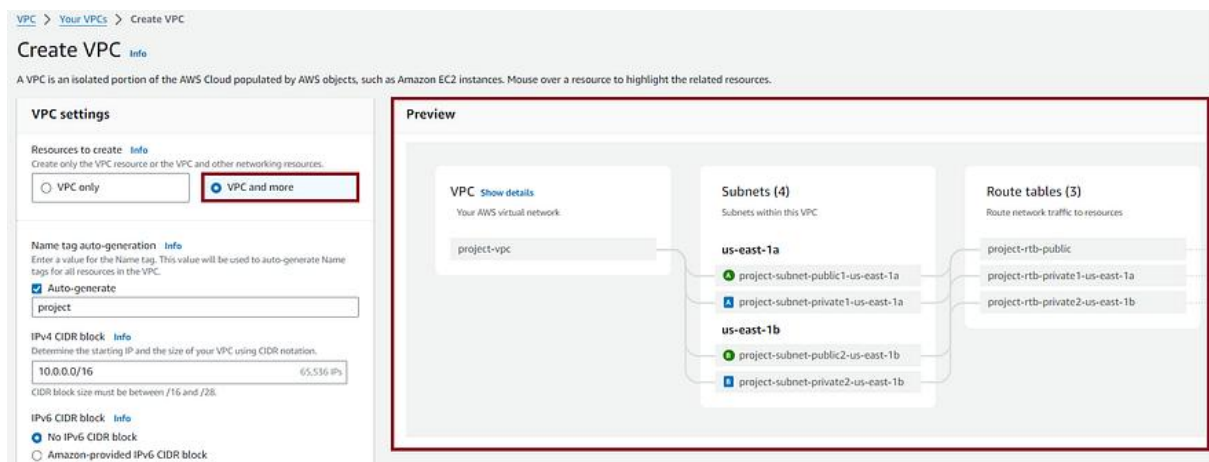
## Step 1: Sign into the AWS Console

Sign in to the AWS Management Console: Log in to your AWS account and navigate to the VPC dashboard.



## Step 2: Create a VPC

After signing in to the AWS Management Console and navigating to the VPC dashboard, click on “Create VPC.” Upon doing so, you will encounter two options under VPC settings:



## VPC Only (Manual Option):

- This option allows you to manually configure the VPC settings, including specifying the IPv4 and optional IPv6 CIDR blocks for your VPC. You will need to define the CIDR block, tenancy (default or dedicated), and optionally enable DNS hostnames and DNS resolution for your VPC. This manual approach provides greater flexibility in customizing your VPC configuration according to your specific requirements.

### **VPC and More (Wizard):**

- The “VPC and More” option launches a wizard that streamlines the process of creating a VPC and its associated components with just one click. This wizard simplifies the VPC creation process by guiding you through the configuration steps for essential components such as subnets, route tables, and internet gateways. It is an efficient way to quickly set up a basic VPC infrastructure without the need for manual configuration.

In this blog we'll go with the manual option for creating the VPC, as it provides more granular control over the configuration process, allowing you to observe firsthand how each component is created within your custom VPC.

- Click on the “VPC Only” option to initiate the manual VPC creation process. This selection indicates that you'll be configuring the VPC settings step by step without the assistance of a wizard.
- Give a Name to Your VPC: Enter “DevOps” as the name for your VPC to easily identify it within your AWS environment.
- IPv4 CIDR Block: Specify the CIDR block as “10.0.0.0/16.” This range encompasses IP addresses from 10.0.0.0 to 10.0.255.255, providing a total of 65,536 IPv4 addresses for your VPC.
- IPv6 CIDR Range: Leave this option unchecked, as we won't be configuring an IPv6 CIDR range for this VPC.
- Tenancy: Keep the tenancy as default, which allows instances launched within the VPC to use shared (default) tenancy unless you specifically require dedicated tenancy for compliance or regulatory reasons.
- After configuring these settings, proceed by clicking on the “Create VPC” button to initiate the creation of your custom VPC named “DevOps” with the specified IPv4 CIDR block.

IPv4 CIDR

10.0.0.0/16

CIDR block size must be between /16 and /28.

IPv6 CIDR block [Info](#)

☒ No IPv6 CIDR block

☐ IPAM-allocated IPv6 CIDR block

☐ Amazon-provided IPv6 CIDR block

☐ IPv6 CIDR owned by me

Tenancy [Info](#)

Default

## Tags

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key

Value - optional

Q Name

X

Q DevOps

X

Remove tag

Add tag

You can add 49 more tags

Cancel

Create VPC

Currently, our custom VPC is devoid of any components. Let's begin the process of creating each component individually.

### Step 3: Configure Subnets

In this step, we will create multiple subnets, comprising both public and private subnets, across two availability zones (us-east-1a and us-east-1b).

Click on Subnets on the left navigation bar and select the custom VPC we created earlier:

[VPC](#) > [Subnets](#) > Create subnet

## Create subnet [Info](#)

### VPC

VPC ID

Create subnets in this VPC.

vpc-01dec7327f2c6d49e (DevOps)

Associated VPC CIDRs

IPv4 CIDRs

10.0.0.0/16

Now under subnet settings, we need to provide the name of the subnet, Availability Zone, and IPv4 subnet CIDR block.

Here is the breakdown of what we will create:

#### Public Subnets:

- Name: Public-1A  
IPv4 CIDR Block: 10.0.1.0/24  
Availability Zone: us-east-1a
- Name: Public-1B  
IPv4 CIDR Block: 10.0.2.0/24  
Availability Zone: us-east-1b

#### Private Subnets:

- Name: Private-1A  
IPv4 CIDR Block: 10.0.3.0/24  
Availability Zone: us-east-1a
- Name: Private-1B  
IPv4 CIDR Block: 10.0.4.0/24  
Availability Zone: us-east-1b

#### Subnet settings

Specify the CIDR blocks and Availability Zone for the subnet.

##### Subnet 1 of 1

###### Subnet name

Create a tag with a key of 'Name' and a value that you specify.

Public-1A

The name can be up to 256 characters long.

###### Availability Zone [Info](#)

Choose the zone in which your subnet will reside, or let Amazon choose one for you.

US East (N. Virginia) / us-east-1a

###### IPv4 VPC CIDR block [Info](#)

Choose the VPC's IPv4 CIDR block for the subnet. The subnet's IPv4 CIDR must lie within this block.

10.0.0.0/16

###### IPv4 subnet CIDR block

10.0.1.0/24

256 IPs

< > ^ v

Similarly, we will repeat for the other three subnets. Once complete click on Create Subnet and you should see something similar to the below:

**Subnets (4)** Info

Find resources by attribute or tag

Subnet ID : subnet-049b0e09931e7e046 X Subnet ID : subnet-096ddceef7827b827 X Subnet ID : subnet-09d47624ee2f9a194 X Show more (+1) Clear filters

| <input type="checkbox"/> | Name       | Subnet ID                                | State     | VPC                                            | IPv4 CIDR   | IPv6 CIDR |
|--------------------------|------------|------------------------------------------|-----------|------------------------------------------------|-------------|-----------|
| <input type="checkbox"/> | Private-1A | <a href="#">subnet-09d47624ee2f9a194</a> | Available | <a href="#">vpc-01dec7327f2c6d49e</a>   Dev... | 10.0.3.0/24 | –         |
| <input type="checkbox"/> | Public-1B  | <a href="#">subnet-096ddceef7827b827</a> | Available | <a href="#">vpc-01dec7327f2c6d49e</a>   Dev... | 10.0.2.0/24 | –         |
| <input type="checkbox"/> | Private-1B | <a href="#">subnet-0579eb97ebf8a4483</a> | Available | <a href="#">vpc-01dec7327f2c6d49e</a>   Dev... | 10.0.4.0/24 | –         |

## Step 4: Configure Route Tables

Now that our subnets have been created it is time to head to the “Route Tables” section in the VPC dashboard.

Upon accessing the Route Tables section, you’ll notice a default route table already associated with our newly created VPC.

In this route table by default, all four subnets that we created appear under their subnet associations as shown below:

Routes Subnet associations Edge associations Route propagation Tags

**Explicit subnet associations (0)** Edit subnet associations

Find subnet association

| Name                                                               | Subnet ID | IPv4 CIDR | IPv6 CIDR |
|--------------------------------------------------------------------|-----------|-----------|-----------|
| No subnet associations<br>You do not have any subnet associations. |           |           |           |

**Subnets without explicit associations (4)** Edit subnet associations

The following subnets have not been explicitly associated with any route tables and are therefore associated with the main route table:

Find subnet association

| Name       | Subnet ID                                | IPv4 CIDR   | IPv6 CIDR |
|------------|------------------------------------------|-------------|-----------|
| Private-1A | <a href="#">subnet-09d47624ee2f9a194</a> | 10.0.3.0/24 | –         |
| Public-1B  | <a href="#">subnet-096ddceef7827b827</a> | 10.0.2.0/24 | –         |
| Private-1B | <a href="#">subnet-0579eb97ebf8a4483</a> | 10.0.4.0/24 | –         |
| Public-1A  | <a href="#">subnet-049b0e09931e7e046</a> | 10.0.1.0/24 | –         |

In our scenario, it’s imperative to segregate the routing for private subnets to prevent them from having direct access to the internet. Additionally, if we intend to create a NAT Gateway for private subnet resources to access the internet, it’s necessary to separate their routing. Therefore, we’ll create a new route table dedicated to the private subnets to facilitate this segregation.

After selecting “Create route table,” a new window will pop up. Here, you’ll need to assign a name to your route table, choose your custom VPC, and then proceed by clicking on “Create route table,” as illustrated in the screenshot below.

VPC > Route tables > Create route table

## Create route table [Info](#)

A route table specifies how packets are forwarded between the subnets within your VPC, the internet, and your VPN connection.

### Route table settings

#### Name - optional

Create a tag with a key of 'Name' and a value that you specify.

devops-private-rt

#### VPC

The VPC to use for this route table.

vpc-01dec7327f2c6d49e (DevOps)

### Tags

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

#### Key

Name

#### Value - optional

devops-private-rt

Remove

Add new tag

You can add 49 more tags.

Cancel

Create route table

Next, under “Explicit subnet associations,” choose “Edit subnet associations.” Here, select both private subnets and then click “Save associations” to ensure they are associated with the newly created route table.

VPC > Route tables > rtb-0eb69bb60bf8f89cf > Edit subnet associations

### Edit subnet associations

Change which subnets are associated with this route table.

#### Available subnets (2/4)

Filter subnet associations

|                                     | Name       | Subnet ID                | IPv4 CIDR   | IPv6 CIDR | Route table ID               |
|-------------------------------------|------------|--------------------------|-------------|-----------|------------------------------|
| <input checked="" type="checkbox"/> | Private-1A | subnet-09d47624ec2f9a194 | 10.0.3.0/24 | -         | Main (rtb-0890225a9b3116319) |
| <input type="checkbox"/>            | Public-1B  | subnet-096ddccef7827b827 | 10.0.2.0/24 | -         | Main (rtb-0890225a9b3116319) |
| <input checked="" type="checkbox"/> | Private-1B | subnet-0579eb97ebf8a4483 | 10.0.4.0/24 | -         | Main (rtb-0890225a9b3116319) |
| <input type="checkbox"/>            | Public-1A  | subnet-049b0e09931e7e046 | 10.0.1.0/24 | -         | Main (rtb-0890225a9b3116319) |

#### Selected subnets

subnet-09d47624ec2f9a194 / Private-1A X subnet-0579eb97ebf8a4483 / Private-1B X

Cancel

Save associations

We have now configured the route table according to our specific requirements, successfully aligning it with our use case.

### Step 5: Create Internet Gateway

Navigate to the “Internet Gateways” section and click on “Create Internet Gateway.” Provide a name for your internet gateway, then proceed by clicking on “Create internet gateway.”

VPC > Internet gateways > Create internet gateway

## Create internet gateway [Info](#)

An internet gateway is a virtual router that connects a VPC to the internet. To create a new internet gateway specify the name for the gateway below.

### Internet gateway settings

**Name tag**  
Creates a tag with a key of 'Name' and a value that you specify.

devops-IGW

### Tags - optional

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

| Key                                        | Value - optional                        |                                       |
|--------------------------------------------|-----------------------------------------|---------------------------------------|
| <input type="text" value="Name"/>          | <input type="text" value="devops-IGW"/> | <input type="button" value="Remove"/> |
| <input type="button" value="Add new tag"/> |                                         |                                       |

You can add 49 more tags.

Cancel

After creating the internet gateway, the next step is to attach it to our custom VPC. To do this, click on “Actions” and then select “Attach to VPC.” Choose your VPC from the dropdown menu, as depicted in the screenshot below.

VPC > Internet gateways > Attach to VPC (igw-0627c97c648c2b039)

## Attach to VPC (igw-0627c97c648c2b039) [Info](#)

### VPC

Attach an internet gateway to a VPC to enable the VPC to communicate with the internet. Specify the VPC to attach below.

**Available VPCs**  
Attach the internet gateway to this VPC.

▶ AWS Command Line Interface command

Cancel

Now that we’ve created the Internet Gateway, it’s essential to ensure that instances in the public subnet can utilize it by adding a route to the Internet. This crucial step will be addressed in the next phase of our setup.

### Step 6: Adding route in the route table

To enable instances in the public subnet to access the internet via the Internet Gateway, we need to add a route in the route table associated with the public subnet. Here’s how to do it:

1. Navigate to the “Route Tables” section in the VPC dashboard.
2. Locate the route table associated with your public subnet.
3. Click on “Edit routes” and then “Add route.”

VPC > Route tables > rtb-0890225a9b3116319 / **devops-public-rt** Actions ▾

**Details** [Info](#)

|                                         |                          |                                   |                        |
|-----------------------------------------|--------------------------|-----------------------------------|------------------------|
| Route table ID<br>rtb-0890225a9b3116319 | Main<br>Yes              | Explicit subnet associations<br>- | Edge associations<br>- |
| VPC<br>vpc-01dec7327f2c6d49e   DevOps   | Owner ID<br>975049886185 |                                   |                        |

[Routes](#) | [Subnet associations](#) | [Edge associations](#) | [Route propagation](#) | [Tags](#)

**Routes (1)** Both ▾ **Edit routes**

| Destination | Target | Status | Propagated |
|-------------|--------|--------|------------|
| 10.0.0.0/16 | local  | Active | No         |

1. In the “Destination” field, enter “0.0.0.0/0” to represent all internet-bound traffic.
2. In the “Target” field, select the Internet Gateway that we previously created and attached to the VPC.
3. Save the route configuration.

VPC > Route tables > rtb-0890225a9b3116319 > Edit routes

**Edit routes**

| Destination                            | Target                                        | Status | Propagated |
|----------------------------------------|-----------------------------------------------|--------|------------|
| 10.0.0.0/16                            | local                                         | Active | No         |
| <input type="text" value="0.0.0.0/0"/> | <input type="text" value="Internet Gateway"/> | -      | No         |

The final adjustment we’ll make is to ensure that the automatic assignment of public IPv4 addresses is enabled for our public subnets. To do this, follow these steps:

1. Go to the “Subnets” section in the VPC dashboard.
2. Locate the public subnets that we created earlier.
3. Click on “Actions” and select “Edit subnet settings.”
4. Ensure that the option “Auto-assign public IPv4 address” is checked.
5. Save the subnet settings.

By enabling this option, instances launched in the public subnets will automatically receive a public IPv4 address, facilitating communication with the internet.



[VPC](#) > [Subnets](#) > [subnet-049b0e09931e7e046](#) > Edit subnet settings

## Edit subnet settings [Info](#)

### Subnet

Subnet ID

 subnet-049b0e09931e7e046

Name

 Public-1A

### Auto-assign IP settings [Info](#)

Enable AWS to automatically assign a public IPv4 or IPv6 address to a new primary network interface for an instance in this subnet.

☒ Enable auto-assign public IPv4 address [Info](#)

☐ Enable auto-assign customer-owned IPv4 address [Info](#)  
Option disabled because no customer owned pools found.

### Step 7: Create Security Group (Optional)

This step is optional and can be created during the instance creation process. However, to ensure consistency across all our upcoming projects, we'll create a security group separately. Here are the necessary steps:

1. Navigate to the "Security Groups" section in the EC2 dashboard.
2. Click on "Create security group."
3. Provide a name and description for the security group.
4. Make sure to select the VPC that we created earlier.
5. Define inbound and outbound rules to control traffic to and from instances associated with this security group. For now, we'll allow inbound traffic on port 22 (SSH) to enable remote access to instances for administration purposes. Outbound traffic will be permitted by default to allow instances to communicate with external resources on the internet.
6. Save the security group.

By creating a dedicated security group, we can enforce consistent network access controls for our instances across multiple projects and environments.

VPC > Security Groups > sg-0f2c85e01f4e2fcfc - DevOps-SG

sg-0f2c85e01f4e2fcfc - DevOps-SG Actions

**Details**

|                                  |                                           |                                                |                                 |
|----------------------------------|-------------------------------------------|------------------------------------------------|---------------------------------|
| Security group name<br>DevOps-SG | Security group ID<br>sg-0f2c85e01f4e2fcfc | Description<br>This is for all DevOps Projects | VPC ID<br>vpc-01dec7327f2c6d49e |
| Owner<br>975049886185            | Inbound rules count<br>1 Permission entry | Outbound rules count<br>1 Permission entry     |                                 |

Inbound rules Outbound rules Tags

**Inbound rules (1)** Manage tags Edit inbound rules

Search

|                          | Name | Security group rule... | IP version | Type | Protocol | Port range | Source    |
|--------------------------|------|------------------------|------------|------|----------|------------|-----------|
| <input type="checkbox"/> | -    | sgr-0771f48dae194156a  | IPv4       | SSH  | TCP      | 22         | 0.0.0.0/0 |

## Step 8: Network Access Control Lists (NACLs)

Network Access Control Lists (NACLs) are another layer of security in AWS VPCs. They act as a firewall for controlling traffic at the subnet level, allowing or denying traffic based on rules you define. NACLs are stateless, meaning that if you allow inbound traffic, you must also explicitly allow the corresponding outbound traffic, and vice versa. They are evaluated before security groups, which are associated with individual instances.

While NACLs offer additional control over traffic flow, they are typically used in scenarios where fine-grained control at the subnet level is necessary. In our current setup, however, we won't be using NACLs. This decision is based on the fact that security groups provide sufficient control over traffic at the instance level, and our architecture does not require the additional complexity that NACLs introduce.